Exercises for Lecture Course on Modeling and System Identification (MSI)
Albert-Ludwigs-Universität Freiburg – Winter Term 2024-2025

Exercise 6: Recursive Least Squares
(to be returned on Dec 9th, 8:15)

Prof. Dr. Moritz Diehl, Katrin Baumgärtner, Jakob Harzer, Dan Wang,
Ashwin Karichannavar, Premnath Srinivasan

In this exercise you will implement a Recursive Least Squares (RLS) estimator and a forward simulation of a differential drive robot with unicycle dynamics. We will apply the RLS algorithm to position data of a 2-DOF movement in the X-Y plane, measured with a sampling time of 0.0159 s.

1. **Recursive Least Squares applied to position data**

   In this task you will implement the Recursive Least Squares (RLS) algorithm in PYTHON and tune the *forgetting factors*. We approximate the position data by a fourth order polynomial in order to obtain a linear-in-the-parameters (LIP) model. You can assume that the noise on the X and Y measurements is independent. The experiment starts at $t = 0\,\text{s}$.

   (a) CODE: Fit a 4-th order polynomial through the data using linear least-squares. Plot the data and the fit for the X- and Y-coordinate.
   Hint: You need one estimator for each coordinate.
   PAPER: Does the fit seem reasonable? Why do you think that is? (1 point)

   (b) CODE: Implement the RLS algorithm as described in the script *(Check section 5.3.1)* to estimate 4-th order polynomials to fit the data. Do not use forgetting factors yet. Plot the result against the data on the same plot as the previous question.
   PAPER: Compare the LS estimator from (a) with the RLS estimator you obtain after processing N measurements. Please give an explanation for your observation. (2 points)

   (c) CODE: Add a forgetting factor α to your algorithm and try different values for α. Plot the results against the data.
   PAPER: How does α influence the fit? What is a reasonable value for α? (1 point)

   (d) PAPER: How can you compute the covariance Σ_p of the position, if you know the covariance of the estimator $\Sigma_{\hat{\theta}}$?
   Hint: For a random variable $\gamma = A\theta$, where A is a matrix, $\text{cov}(\gamma) = A\text{cov}(\theta)A^{\text{T}}$. (1 point)

   (e) CODE: Compute the *one-step-ahead* prediction at each point (i.e. extrapolate your polynomial fit to the next time step). We also provided code to plot the 1-σ confidence ellipsoid around this point, and the data.
   PAPER: Do the confidence ellipsoids grow bigger or smaller as you take more measurements? Why do you think that is? (2 points)

2. **Covariance approximation**
   Consider a nonlinear function $f : \mathbb{R}^n \to \mathbb{R}$ that maps a random vector $X = (X_1, \ldots, X_n)^\top$ to a scalar random variable Y, i.e.

$$Y = f(X) = f(X_1, \ldots, X_n).$$

   We have $\mathbb{E}\{X\} = \mu_x = (\mu_1, \ldots, \mu_n)^\top$ and $\text{cov}(X) = \Sigma_x \in \mathbb{R}^{n \times n}$.

(a) PAPER: Give an approximation of the expected value $\mathbb{E}\{Y\}$ and the covariance matrix $\text{cov}(Y)$ of Y using a first order Taylor expansion of f around μ_x. (2 points)

(b) PAPER: Suppose $X_1, \ldots, X_n$ are independent. Simplify your covariance approximation from part (a). (1 point)

This sheet gives in total 10 points

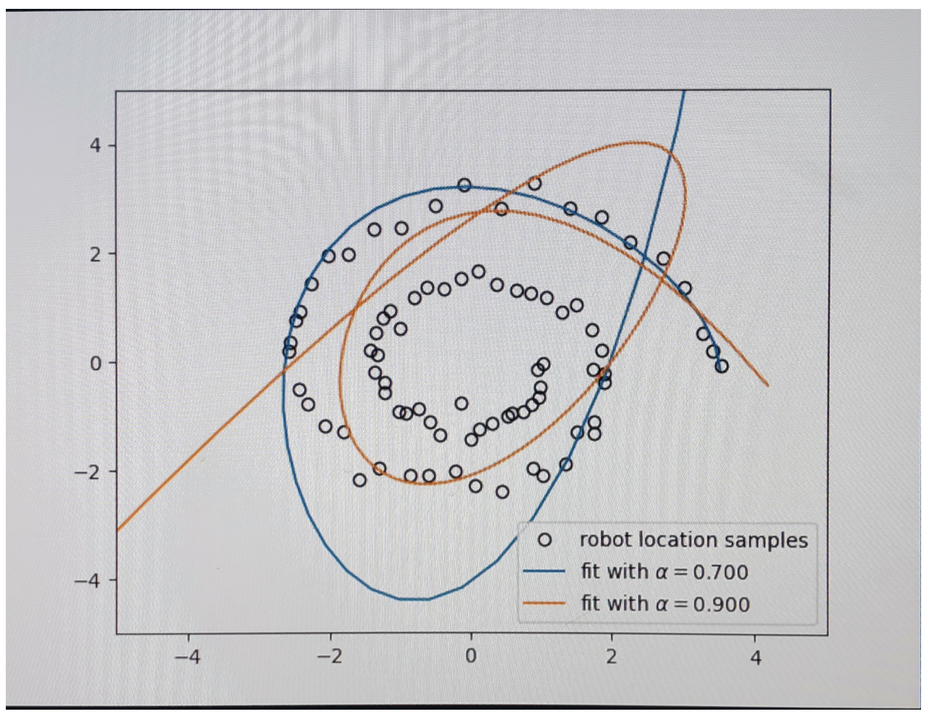
a) $\Phi = \begin{bmatrix} 1 & t & t^2 & t^3 & t^4 \\ 1 & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ t & t^2 & t^3 & t^4 \end{bmatrix}$

LLS solution: $\theta^* = (\Phi^T \Phi)^{-1} \cdot \Phi^T \cdot y_N$

- The fit is not good. Because:
  - the polynomial cannot follow the datapoints leading to degrade in the final performance measurement of x-y.

b). The same as the LLS, because the process of this recursive estimator do not "forget" the covariance, yeilding the "over confidence" of the estimator. Therefore, the final result is similar to the Q(a).

c). The increase of α allows the estimator to fit better with the inside curves and vice versa.



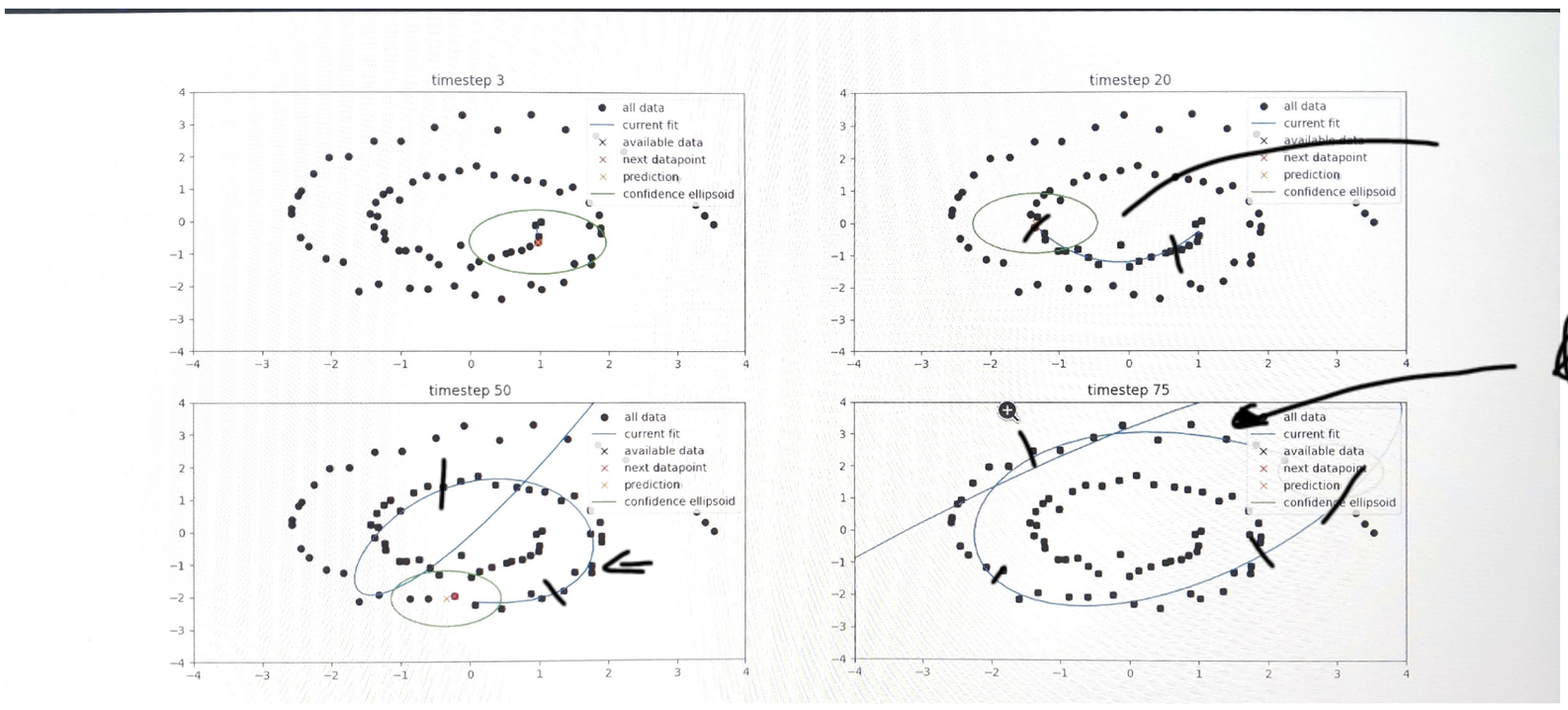
However, The forgeting factor α only is not enough for the estimator to estimate the path of the robot.

d) $y = A\theta \rightarrow Cov(y) = A\,Cov(\theta)\,A^T$.

$$X_{k+1} = \varphi_{k+1}^T \hat{\theta}_k$$

$$\Rightarrow Cov(X_{k+1}) = \varphi_{k+1}^T \cdot Cov(\hat{\theta}_k) \cdot \varphi_{k+1}$$

e)



- If the given $\alpha = \begin{bmatrix} \alpha_x \\ \alpha_y \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, no forgetting factors, the confidence ellipsoid will decrease and disappear when the number of measurements increase.

- However, in the current results (figure above) show that the uncertainties of the estimator are still large, even though the number of itterations increase. There are 2 reasons for this:

  1. $\alpha < 1$ → the estimator continuously shrinks the information matrix Q, preventing it from reaching 100% confidence.

  2. The mismatch of the model: the 4th polynomial equation cannot adequately represent the "spiral path" of the robot. For that reason, the proposed (4th) model can only fit certain region of robot trajectory (local area) before shooting off to infinity.

2) a. $f : R^n \rightarrow R$

Taylor expansion of $f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!}(x-a)$

$$\Rightarrow f(x) = f(\mu_x) + \frac{f'(\mu_x)}{1!}(x-\mu_x) + \frac{f''(\mu_x)}{2!}(x-\mu_x)^2 + \mathcal{O}(3)$$

$$\rightarrow \boxed{Y \approx f(\mu_x) + \nabla f(\mu_x)^T \cdot (x-\mu_x)}$$ only cares about 1st order.

$$E\{Y\} = E\{f(\mu_x) + \nabla f(\mu_x)^T \cdot (x-\mu_x)\}$$

$$= E\{f(\mu_x)\} + E\{\nabla f(\mu_x)^T \cdot (x-\mu_x)\}$$

$$= f(\mu_x) + E\{\nabla f(\mu_x)^T\} \cdot E\{(x-\mu_x)\}$$

$$= f(\mu_x) + \nabla f(\mu_x)^T \cdot (\underbrace{E\{x\}}_{\mu_x} - E\{\mu_x\})$$

$$= f(\mu_x)$$

- Adding / substracting by constants → Does not change the spread of Data.
- Multiplying / Dividing by constants → Does change the spread of Data.

$$Cov(Y) = cov(\underline{f(\mu_x)} + \nabla f(\mu_x)^T (x-\mu_x))$$

$$= cov(\underbrace{f(\mu_x)}_{\text{const.}} + \nabla f(\mu_x)^T \cdot x - \underbrace{\nabla f(\mu_x)^T \cdot \mu_x}_{\text{const.}})$$

$$cov(y) \approx cov(\underbrace{\nabla f(\mu_x)^T}_{A} \cdot x)$$

$$cov(A\theta) = A \cdot cov(\theta) \cdot A^T$$

$$= \nabla f(\mu_x)^T \cdot cov(x) \cdot \nabla f(\mu_x)$$

$$= \boxed{\nabla f(\mu_x)^T \cdot \Sigma_x \cdot \nabla f(\mu_x)}$$

b) $X_1 \dots, X_n$ are independent.

$$\Rightarrow \Sigma_x = \begin{bmatrix} \sigma_{x_1}^2 & 0 & 0 \\ 0 & \sigma_{x_2}^2 & 0 \\ 0 & 0 & \sigma_{x_n}^2 \end{bmatrix}$$

- $\nabla f(\mu_x) = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]^T$

$$\Rightarrow Cov(Y) = \underbrace{\begin{bmatrix} \frac{\partial f}{\partial x_1} & \frac{\partial f}{\partial x_2} & \dots & \frac{\partial f}{\partial x_n} \end{bmatrix}}_{(1 \times n)} \underbrace{\begin{bmatrix} \sigma_{x_1}^2 & 0 & 0 \\ 0 & \sigma_{x_2}^2 & 0 \\ & & \ddots \\ 0 & 0 & \sigma_{x_n}^2 \end{bmatrix}}_{(n \times n)} \underbrace{\begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}}_{(n \times 1)}$$

$$= \boxed{\sum_{i=1}^{n} \left(\frac{\partial f}{\partial x_i} \cdot \sigma_{x_i} \right)^2} = \text{Error Propagation Formula.}$$